VIT
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING (SCOPE)

BCSE406L – NoSQL Databases | Fall Semester 2026–2027

DIGITAL ASSIGNMENT – 3 | Final Project Submission Report

VERITAS

An Evidence-Aware, Temporal and Explainable NoSQL Database for Evolving AI Debates

End-to-End Document Database Architecture, Multi-Factor Evidence Scoring, Contradiction Detection, Temporal State Auditing, and Interactive Graph Visualization over MongoDB 7

Academic Submission Field	Official Details
Individual Submission By	**SIDDHESH SHINDE**
Register Number	**23BCE5048**
Individual Project Role	**Frontend, AI Analysis Pipeline, Multi-Factor Scoring Engine & Visualization Lead**
Course Code & Title	BCSE406L – NoSQL Databases
Semester & Academic Year	Fall Semester 2026–2027
Slot & Class ID	Slot: D2+TD2 | Class: CH202670101354
Faculty Evaluator	Dr. S. Geetha, School of Computer Science and Engineering, VIT Chennai
Project Team Roster	1. Naman Sharma (23BCE1683) — MongoDB Data Architect & Backend Lead 2. Megh (23BCE1665) — FastAPI Backend Routes, CRUD & QueryService Developer 3. Siddhesh Shinde (23BCE5048) — Scoring Engine, Temporal History, AI Pipeline & Frontend Lead
GitHub Source Repository	https://github.com/siddheshshindeofficial/VERITAS-main

1. Final Project Overview, Abstract & Problem Statement

1.1 Abstract

VERITAS (*An Evidence-Aware, Temporal and Explainable Database for Evolving AI Debates*) is a full-stack NoSQL web application that models multi-perspective analytical debates as persistent, evolving document graphs in MongoDB 7 (`veritas_db`) rather than ephemeral conversational text. When users evaluate complex policy, medical, or technological questions, standard large language model responses flatten competing claims into a single static paragraph without preserving explicit links between individual assertions, supporting or opposing empirical evidence, counterarguments, pairwise contradictions, and chronological confidence shifts.

Built with a React 19 and TypeScript frontend, a Python 3.12 and FastAPI REST service tier, and PyMongo 4 over MongoDB 7, VERITAS decomposes every debate into eight active document collections connected via hybrid subdocument embedding and `ObjectId` referencing. The system implements a deterministic four-factor evidence scoring algorithm, a claim-strength aggregation calculator, a Jaccard token-overlap contradiction detector, an append-only temporal event log, 12 analytical MongoDB queries (`Q1`–`Q12`), 5 multi-stage aggregation pipelines (`AGG-1`–`AGG-5`), and 8 custom compound and single-field indexes. Full source code, seed scripts, and automated tests are published at https://github.com/siddheshshindeofficial/VERITAS-main.

1.2 Problem Statement & Research Motivation

In research, clinical governance, and public policy analysis, decision-makers routinely investigate questions where credible evidence exists on both sides—for example, *"Should AI be used for medical diagnosis?"* or *"Should cities impose stricter restrictions on private vehicles?"* Traditional relational databases require rigid, normalized join tables to link claims, polymorphic provenance metadata, and multi-hop rebuttals, making schema evolution brittle. Conversely, stateless AI chat interfaces discard the structural graph connecting a claim to its underlying evidence and provide no mechanism to query how a debate's balance of confidence shifted as new evidence arrived.

VERITAS solves this problem by elevating the debate itself into a persistent NoSQL knowledge object. Every claim, piece of evidence, bibliographic source, rebuttal, contradiction, and historical synthesis snapshot is stored as an independently inspectable BSON document in MongoDB, enabling complete traceability along the flow: **Application → Database → Query/Aggregation → Result**.

1.3 Project Objectives & Scope

1. **Document-Oriented Debate Modeling:** Design a hybrid MongoDB schema in `veritas_db` that embeds frequently co-read synthesis state inside root `debates` documents while referencing high-cardinality entities (`claims`, `evidence`, `counterarguments`, `contradictions`, `sources`, `debate_history`, and `embeddings`) via `ObjectId` links.
2. **Deterministic Evidence & Claim Scoring Engine:** Implement reproducible Python mathematical models (`scoring.py`, `claim_strength.py`) that compute evidence scores from Reliability, Relevance, Independence, and Recency, and roll them up into PRO/CON debate confidence and uncertainty metrics.
3. **Automated Conflict & Temporal Tracking:** Detect potential conflicts between PRO and CON claims using lexical Jaccard similarity and polarity heuristics (`contradiction_engine.py`), and record append-only state snapshots in `debate_history` (`temporal.py`).

4. **Expressive MongoDB Queries, Aggregations & Indexing:** Implement 12 project-specific queries (`Q1`–`Q12`), 5 aggregation pipelines (including cross-collection `$lookup` joins), and 8 access-pattern-aligned indexes.
5. **Interactive Full-Stack Visualization:** Deliver a React 19 interface with React Flow argument graphs, Recharts temporal evolution timelines, a live Query Explorer, an Admin index inspector, and a Database Health dashboard.

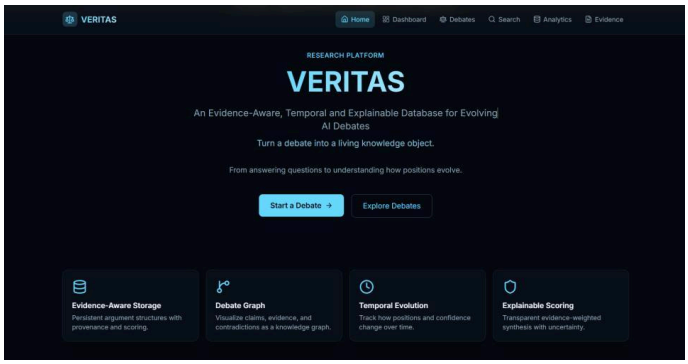


Figure 1.1: VERITAS web application landing page highlighting Evidence-Aware Storage, Debate Graph, Temporal Evolution, and Explainable Scoring.

2. Justification for NoSQL Model & Final System Architecture

2.1 Why MongoDB & the Document Model Fit Evolving Debate Graphs

An evolving debate exhibits three data characteristics that make a document-oriented NoSQL database superior to a traditional third-normal-form relational schema:

- **Hierarchical Locality for Dashboard & Workspace Reads:** The root debate question and its current synthesis summary (`pro_score`, `con_score`, `confidence`, `uncertainty`, and text summary) are always displayed together. Embedding `current_synthesis` inside each `debates` document allows a single `findOne()` read to populate the workspace header with zero join latency.
- **Schema Flexibility for Polymorphic Provenance:** Claims and evidence records originate from diverse workflows—manual user entry, seeded domain fixtures, or automated AI analysis pipelines. Consequently, their `provenance` and `metadata` subdocuments carry varying keys (such as `generated_by`, `source_context`, or detailed `strength_explanation` breakdowns) that MongoDB stores seamlessly without `ALTER TABLE` migrations.
- **Native Pipeline Aggregations & Vector Storage:** MongoDB supports multi-stage aggregation pipelines (`$match`, `$group`, `$lookup`, `$addFields`, `$map`, `$cond`) alongside native array storage for vector embeddings (`embeddings.embedding[]`), allowing analytical rollups and semantic similarity retrieval within a single unified data store.

2.2 End-to-End Three-Tier System Architecture

As shown in Figure 2.1, the React 19 frontend communicates with the FastAPI backend over RESTful JSON endpoints. FastAPI validates incoming payloads using Pydantic v2 models, invokes modular domain services (Scoring, Contradiction Detection, Temporal History, Search, and AI Analysis), executes BSON queries against MongoDB 7 via a shared PyMongo client, and serializes `ObjectId` and ISO-8601 `datetime` fields for client consumption. TanStack Query manages client-side caching and automatic query invalidation whenever mutations occur.

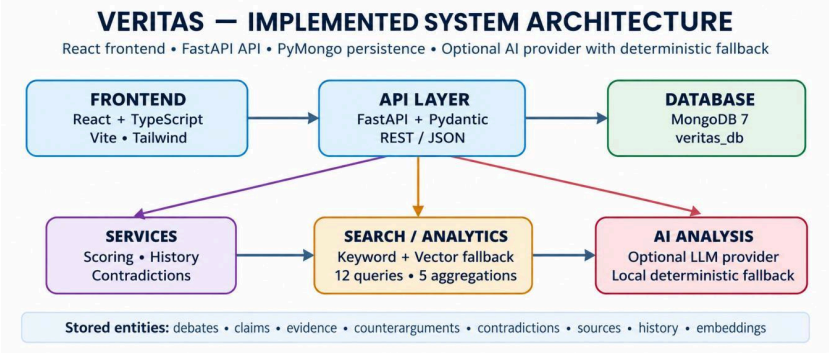


Figure 2.1: Implemented three-tier architecture of VERITAS connecting React 19, FastAPI domain services, and MongoDB 7 (veritas_db).

2.3 Technology Stack & Local Execution Commands

Architectural Tier	Technologies & Libraries	Implementation Responsibilities
Frontend Layer	React 19, TypeScript, Vite 8, Tailwind CSS 4, React Router, TanStack Query	Provides 10 interactive application views, form validation (React Hook Form + Zod), and asynchronous state synchronization.
Visualization Layer	React Flow, Recharts, Lucide Icons	Renders interactive node-link debate graphs (claims, evidence, rebuttals) and time-series confidence charts.
Backend API Layer	Python 3.12, FastAPI 0.115, Pydantic 2, Uvicorn	Implements REST CRUD routes, OpenAPI/Swagger documentation, scoring services, and aggregation endpoints.
Database Layer	MongoDB 7 (Docker Compose `mongo:7`), PyMongo 4	Stores `veritas_db` across 8 active collections with 16 total indexes (8 custom + 8 primary `_id`).
AI & Search Layer	OpenAI-Compatible Adapter + Deterministic Local Fallback	Executes structured debate analysis and synthesis generation; operates offline via deterministic domain templates when no external API key is set.
Testing & QA	Pytest Test Suite (`backend/tests/test_api.py`)	Verifies 11 end-to-end API, CRUD, aggregation, and database connectivity test cases.

```
# Terminal 1: Launch MongoDB 7 service in Docker
docker compose up -d mongodb

# Terminal 2: Initialize indexes, seed dataset, and start FastAPI backend
(http://127.0.0.1:8000)
cd backend
.\venv\Scripts\python.exe scripts/indexes.py
.\venv\Scripts\python.exe scripts/seed.py
.\venv\Scripts\python.exe -m uvicorn app.main:app --reload --host 127.0.0.1 --port 8000

# Terminal 3: Start React 19 Vite frontend server (http://127.0.0.1:5173)
cd frontend
npm.cmd run dev -- --host 127.0.0.1
```

3. Final NoSQL Database Design & Document Schemas

3.1 Collection Catalogue & Relationship Strategy

The `veritas_db` database defines nine collections: eight actively populated collections that power all application workflows and one reserved collection (`audit_logs`) for administrative compliance extensions. Figure 3.1 depicts the entity relationships across the database.

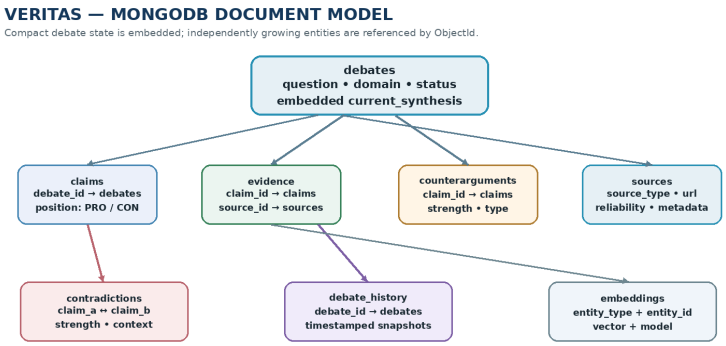


Figure 3.1: Final MongoDB document model showing embedded synthesis state and ObjectId cross-collection references.

Collection	Design Pattern	Key Attributes & Referential Integrity
`debates`	Root Object + Embedded Subdocuments	`_id`, `question`, `domain`, `description`, `status`, `created_at`, `updated_at`, embedded `current_synthesis`, and `metadata`.
`claims`	1:N Referenced from `debates`	`_id`, `debate_id` (ref `debates._id`), `position` (`PRO` | `CON`), `text`, `normalized_text`, `confidence`, `status`, `tags[]`, `evidence_ids[]`, `counterargument_ids[]`, `provenance`.
`evidence`	1:N Referenced from `claims` & `sources`	`_id`, `claim_id` (ref `claims._id`), `source_id` (ref `sources._id`), `title`, `content`, `source_type`, `supports`, `relevance`, `reliability_score`, `independence_score`, `recency_score`, `evidence_score`.
`counterarguments`	1:N Referenced from `claims`	`_id`, `claim_id` (ref `claims._id`), `text`, `strength`, `type` (`rebuttal` | `qualification` | `counterclaim`), `generated_by`, `created_at`.
`contradictions`	M:N Pairwise Reference between `claims`	`_id`, `claim_a` (ref `claims._id`), `claim_b` (ref `claims._id`), `strength`, `type`, `explanation`, `context_dimensions`, `created_at`.
`sources`	Shared Referenced Catalogue	`_id`, `title`, `authors[]`, `publisher`, `source_type`, `url`, `published_at`, `reliability`, `metadata`.
`debate_history`	1:N Append-Only Time-Series	`_id`, `debate_id` (ref `debates._id`), `timestamp`, `pro_score`, `con_score`, `confidence`, `evidence_count`, `contradiction_count`, `event_type`, `change_summary`.
`embeddings`	Polymorphic Vector Store	`_id`, `entity_type` (`claim` | `evidence` | `counterargument`), `entity_id` (ref target `_id`), `embedding[]`, `model`, `created_at`.

Collection	Design Pattern	Key Attributes & Referential Integrity
`audit_logs`	Reserved Security Trail	Registered in health monitoring for administrative write auditing.

3.2 Hybrid Embedding vs. Referencing Rationale

Architectural Decision	Engineering & Performance Justification
Embed `current_synthesis` and `metadata` in `debates`	Bounded 1:1 subdocuments that are read on every dashboard card and workspace header load; embedding eliminates extra network round-trips.
Reference `claims` via `claims.debate_id`	A debate can accumulate dozens of PRO and CON claims that are queried, filtered, ranked (Q2, Q3, Q9, Q10), and updated independently.
Reference `evidence` via `claim_id` and `source_id`	Evidence items carry independent four-factor scores and link to reusable bibliographic `sources`; storing them separately enables direct source-type aggregations (Q7) and `$lookup` joins (Q11).
Store `contradictions` as pairwise link documents	A contradiction represents a symmetric edge between two distinct claims (`claim_a` and `claim_b`), which is naturally modeled as a dedicated edge collection.
Store `debate_history` and `embeddings` separately	Append-only event snapshots and high-dimensional float vectors grow continuously; isolating them prevents root document bloat and keeps working-set BSON sizes compact.

3.3 Production BSON Document Schemas

```
// 1. debates document (Healthcare Debate: _id 6aaeee47bba2e3c0b46e2e74)
{
  "_id": ObjectId("6aaeee47bba2e3c0b46e2e74"),
  "question": "Should AI be used for medical diagnosis?",
  "domain": "Healthcare",
  "description": "Examining AI-assisted diagnostic tools in clinical settings.",
  "status": "active",
  "created_at": ISODate("2026-09-10T20:19:19.684Z"),
  "updated_at": ISODate("2026-09-10T20:19:19.684Z"),
  "current_synthesis": {
    "summary": "Multi-claim synthesis balancing clinical triage gains against validation
risks.",
    "confidence": 0.6175,
    "pro_score": 0.625,
    "con_score": 0.61,
    "uncertainty": 0.985
  },
  "metadata": { "created_by": "seed_script", "language": "en", "development_fixture": true }
}

// 2. claims document (_id 6aaeee47bba2e3c0b46e2e76)
{
  "_id": ObjectId("6aaeee47bba2e3c0b46e2e76"),
  "debate_id": ObjectId("6aaeee47bba2e3c0b46e2e74"),
  "position": "PRO",
  "text": "Evidence suggests significant benefits in Healthcare contexts when properly
```

```
validated.",
  "normalized_text": "evidence suggests significant benefits in healthcare contexts when
pro...",
  "confidence": 0.6,
  "status": "active",
  "tags": ["healthcare"],
  "evidence_ids": [ObjectId("6aaeee47bba2e3c0b46e2e78"),
ObjectId("6aaeee47bba2e3c0b46e2e79")],
  "counterargument_ids": [ObjectId("6aaeee47bba2e3c0b46e2e7a")],
  "provenance": { "generated_by": "retrieval", "source_context": "seed_script" }
}

// 3. evidence document (_id 6aaeee47bba2e3c0b46e2e78)
{
  "_id": ObjectId("6aaeee47bba2e3c0b46e2e78"),
  "claim_id": ObjectId("6aaeee47bba2e3c0b46e2e76"),
  "source_id": ObjectId("6aaeee47bba2e3c0b46e2e5e"),
  "title": "Fixture evidence for PRO claim 1",
  "source_type": "research_paper",
  "supports": true,
  "relevance": 0.55,
  "reliability_score": 0.5,
  "independence_score": 0.55,
  "recency_score": 0.6,
  "evidence_score": 0.0908,
  "published_at": ISODate("2026-07-22T20:19:19.684Z")
}
```

3.4 Live MongoDB Compass & Docker Verification

Figures 3.2 through 3.6 show the live MongoDB 7 instance running inside Docker Desktop (`veritas-mongo`) and the populated collections and BSON records verified inside MongoDB Compass. Across the populated database state shown in Compass, `veritas_db` contains 11 debates, 52 claims, 100 evidence items, 52 counterarguments, 29 contradictions, 44 sources, 67 history snapshots, and 152 vector embeddings (507 total documents).

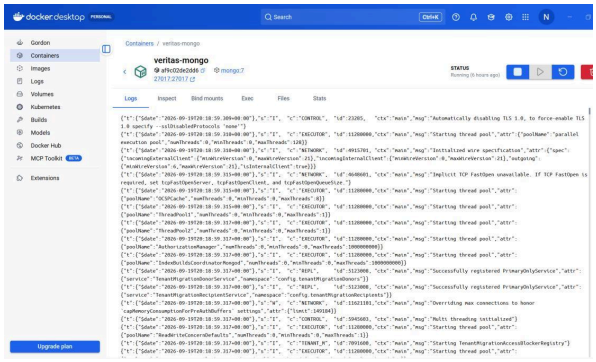


Figure 3.2: Docker Desktop showing the running veritas-mongo (mongo:7) container and active WiredTiger logs on port 27017.

Once the container is active, connecting MongoDB Compass to `mongodb://localhost:27017` exposes the eight populated collections in `veritas_db` (Figure 3.3), along with storage metrics, average document sizes, and the two indexes registered on each collection.

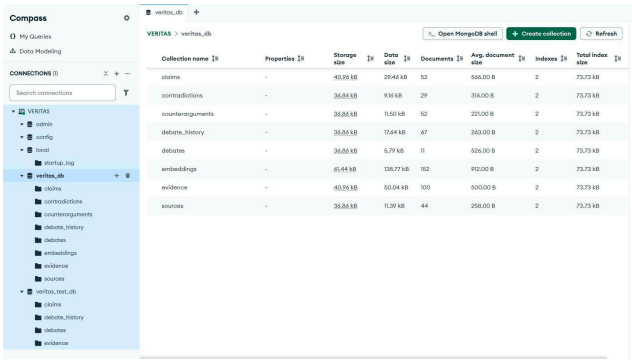


Figure 3.3: MongoDB Compass workspace displaying the 8 populated collections in veritas_db, document counts (507 total), and 16 indexes.

Expanding individual collections in MongoDB Compass confirms the exact field types, nested objects, ISO timestamps, and `ObjectId` foreign keys across `debates` (Figure 3.4), `claims` (Figure 3.5), and `evidence` (Figure 3.6).

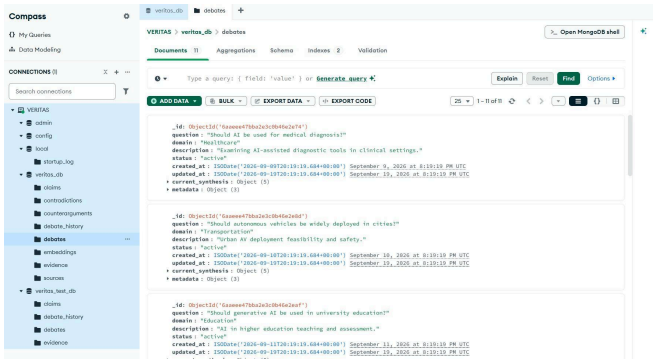


Figure 3.4: MongoDB Compass inspecting root debate documents in veritas_db.debates across Healthcare, Transportation, and Education domains.

In Figure 3.5, the `veritas_db.claims` view in MongoDB Compass demonstrates how individual PRO and CON claim documents reference their parent debate via `debate_id` and maintain fast navigation caches (`evidence_ids`, `counterargument_ids`) alongside provenance metadata.

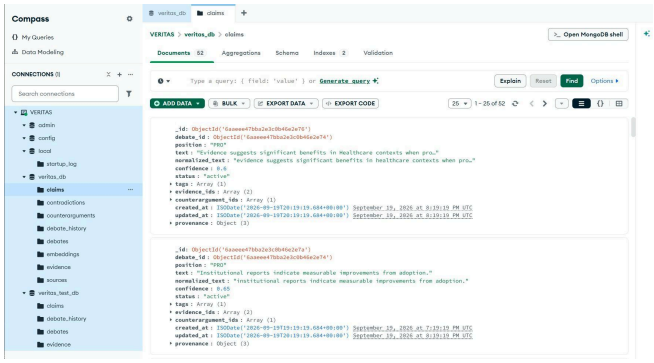


Figure 3.5: MongoDB Compass displaying PRO claim documents in veritas_db.claims with confidence scores and linked evidence_ids arrays.

In Figure 3.6, the `veritas_db.evidence` collection view confirms that each evidence record stores its four component evaluation dimensions (`relevance`, `reliability_score`, `independence_score`, and `recency_score`) and the resulting multiplicative `evidence_score`.

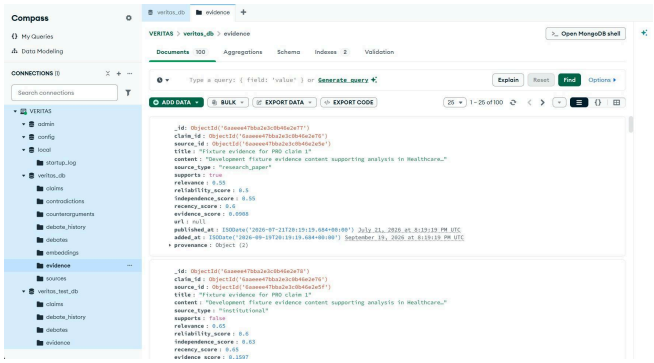


Figure 3.6: MongoDB Compass showing supporting and opposing evidence records in veritas_db.evidence with computed evidence_score values.

4. Scoring Engine, Contradiction Detection, Temporal History & AI Pipeline (Implemented by Siddhesh Shinde – 23BCE5048)

4.1 Multi-Factor Evidence & Claim-Strength Scoring Engine

As the AI, Scoring, and Frontend Visualization Lead (**Siddhesh Shinde, 23BCE5048**), I designed and implemented the mathematical scoring services in `backend/app/services/scoring.py` and `backend/app/services/claim_strength.py`. Rather than relying on black-box LLM confidence guesses, every evidence record is evaluated across four bounded inputs in the interval `[0.0, 1.0]`:

```
Evidence Score = round(clip(Reliability) × clip(Relevance) × clip(Independence) ×
clip(Recency), 4)
```

Because the formula is multiplicative, a weak value on any single dimension (such as low source independence or outdated publication recency) penalizes the composite score proportionally. For example, an evidence item with `Reliability = 0.50`, `Relevance = 0.55`, `Independence = 0.55`, and `Recency = 0.60` yields `0.50 × 0.55 × 0.55 × 0.60 = 0.09075`, stored as `0.0908`.

At the claim level (`claim_strength.py`), a claim's confidence is recalculated whenever supporting/opposing evidence, counterarguments, or contradictions are attached. Supporting evidence contributes positively, opposing evidence subtracts a `0.5×` weighted penalty, counterarguments apply a scaled rebuttal deduction, and active contradictions apply a conflict penalty. The resulting score is clamped to `[0.0, 1.0]` and persisted alongside a structured `strength_explanation` subdocument. At the root debate level, `pro_score` and `con_score` represent the mean confidence of active (non-superseded) PRO and CON claims, while `uncertainty` measures the closeness of competing stances (for instance, in the Healthcare debate, `pro_score = 0.625` and `con_score = 0.610` yield a high uncertainty of `0.985`).

4.2 Heuristic Contradiction Detection & Temporal State Service

The contradiction detector (`backend/app/services/contradiction_engine.py`) scans pairs of opposing PRO and CON claims within a debate using normalized token-set Jaccard overlap combined with polarity/antonym heuristics. Identified conflicts are classified into `direct_contradiction`, `contextual_disagreement`, or `potential_conflict`, assigned a numeric `strength` and human-readable `explanation`, and deduplicated before insertion into `veritas_db.contradictions`.

Whenever a debate is created, a claim is added or challenged, a contradiction is detected, or an analysis synthesis completes, `backend/app/services/temporal.py` appends an immutable snapshot document to

`veritas_db.debate_history` containing the timestamp, event type (`debate_created`, `claim_challenged`, `contradiction_detected`, `confidence_updated`, `synthesis_updated`), and current score tuple.

4.3 AI Debate Analysis Lifecycle & Deterministic Fallback

Triggering `POST /api/ai/analyze` executes the eight-stage orchestration workflow implemented in `backend/app/agents/debate_pipeline.py` (illustrated in Figure 4.1):



Figure 4.1: Eight-stage execution lifecycle of the VERITAS debate analysis and scoring pipeline.

1. Load the target debate document from `veritas_db.debates` by `ObjectId`.
2. Generate structured PRO and CON claims if the debate currently has no claims.
3. Synthesize supporting/opposing evidence records and counterarguments linked to each claim.
4. Run pairwise PRO vs. CON comparison via `contradiction_engine.py` and persist new conflicts.
5. Recompute multiplicative `evidence_score` values, claim confidences, and debate-level PRO/CON scores.
6. Execute structured synthesis via the configured OpenAI-compatible provider (or deterministic template fallback when running offline).
7. Update the embedded `debates.current_synthesis` subdocument and append an `analysis_completed` snapshot to `debate_history`.
8. Return updated entity counts and provider mode to the React client, triggering TanStack Query cache invalidation.

5. Application–Database Connectivity, CRUD Operations & API Evidence

5.1 Principal REST CRUD Routes & Frontend Route Architecture

Table 5.1 maps the core backend CRUD controllers in `backend/app/routes/` and the ten React Router views in `frontend/src/pages/`. Figure 5.1 shows the debate browser loading persisted debates from `GET /api/debates`.

Module / Route	HTTP Endpoints / UI Path	Database Interaction & Behavior
Debates CRUD	`POST, GET /api/debates` `GET, PUT, DELETE /api/debates/{id}`	Inserts new debates with initial synthesis, filters by domain/status, updates status, and cascades deletion across debate-owned records.
Claims CRUD	`POST, GET, PUT, DELETE /api/claims` `GET /api/claims/debate/{id}`	Creates PRO/CON claims, recalculates claim and debate scores, and records history events.
Evidence & Sources	`POST, GET, PUT, DELETE /api/evidence` `POST, GET /api/sources`	Computes `evidence_score` on insert, links `source_id`, and updates parent claim `evidence_ids[]`.

Module / Route	HTTP Endpoints / UI Path	Database Interaction & Behavior
Search & AI Analysis	`GET /api/search`, `POST /api/ai/analyze`	Executes case-insensitive regex and cosine vector similarity search; runs full debate analysis pipeline.
React Frontend Views	`/`, `/dashboard`, `/debates`, `/debates/new`, `/debates/:id`, `/search`, `/admin`, `/query-explorer`, `/database-health`, `/projects/evidence`	Provides end-to-end interactive interfaces for debate creation, graph exploration, live query execution (Q1–Q12), index inspection, and rubric verification.

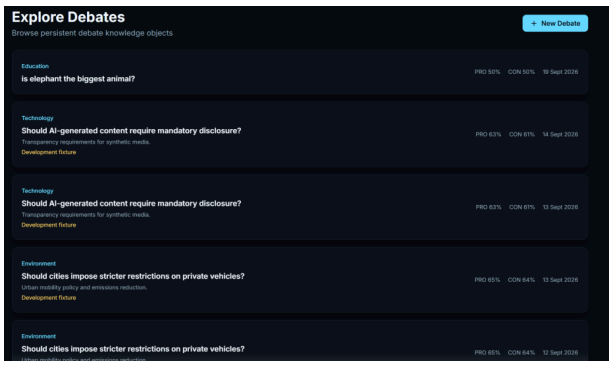


Figure 5.1: Explore Debates page rendering persisted debate documents with PRO/CON confidence ratios and domain badges.

5.2 Swagger UI (OpenAPI) CRUD Execution Evidence

Figures 5.2 through 5.5 demonstrate live execution of all four CRUD operations (Create, Read, Update, and Delete) through FastAPI's interactive Swagger UI at `http://127.0.0.1:8000/docs`.

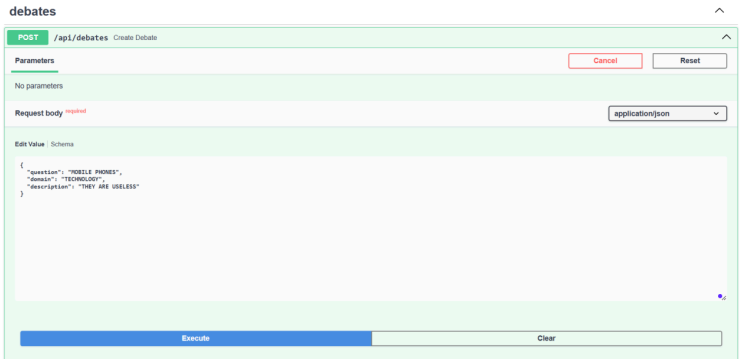


Figure 5.2: CREATE — POST /api/debates inserting a new Technology debate document into veritas_db.debates.

Following creation, Figure 5.3 demonstrates the `GET /api/debates/{debate_id}` endpoint fetching the persisted Healthcare debate document (`6aaeee47bba2e3c0b46e2e74`) along with its embedded synthesis object.

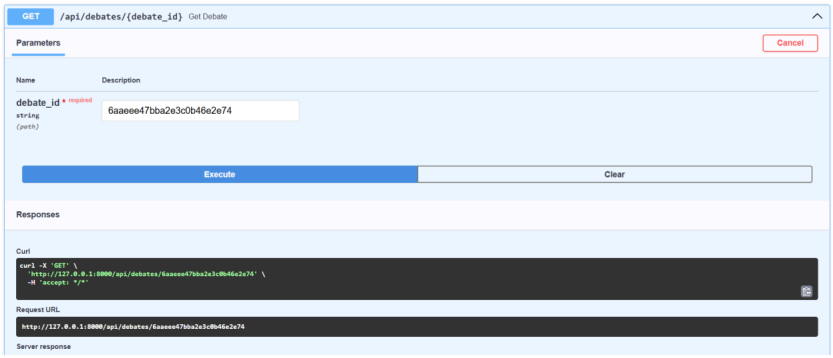


Figure 5.3: READ — GET /api/debates/{debate_id} retrieving the Healthcare debate (6aaeee47bba2e3c0b46e2e74) by ObjectId.

To verify non-destructive mutation and deletion without altering the baseline seeded debates used by analytical queries, Figures 5.4 and 5.5 show updating a test debate's status to `Completed` and subsequently removing the test record (`6aaef7c0e6ab4eed455b0053`) via `DELETE /api/debates/{debate_id}`.

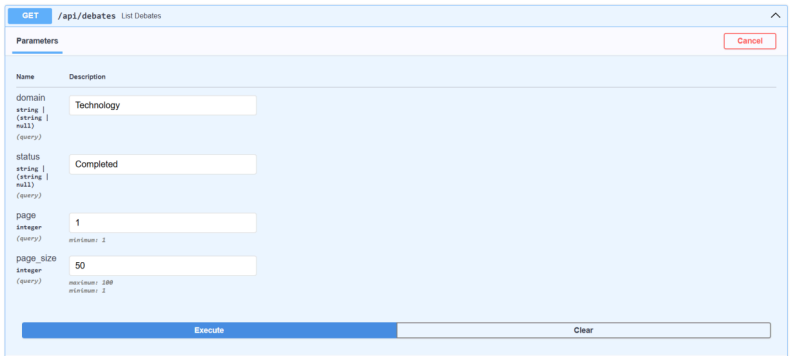


Figure 5.4: UPDATE & FILTER — verifying status transition to Completed via FastAPI Swagger UI.

After verifying the status update, Figure 5.5 shows the execution of `DELETE /api/debates/{debate_id}` on the disposable record (`6aaef7c0e6ab4eed455b0053`), confirming cascading cleanup of associated claims, evidence, counterarguments, and history snapshots.

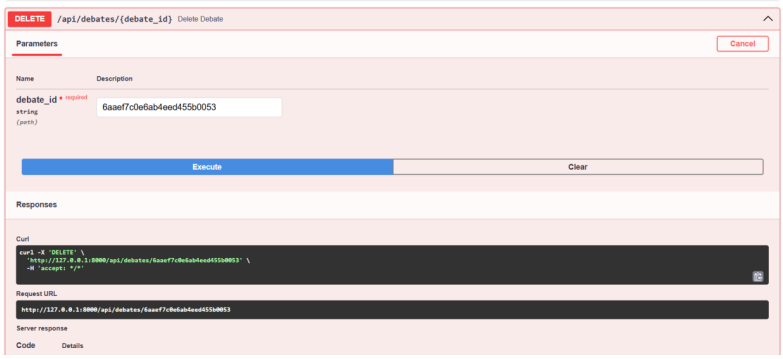


Figure 5.5: DELETE — executing DELETE /api/debates/6aaef7c0e6ab4eed455b0053 and removing linked child records.

6. Important MongoDB Queries (Q1–Q12) & Live Execution Outputs

6.1 Complete Catalogue of 12 Project-Specific Queries

All 12 analytical queries required by the course rubric are implemented in `backend/app/analytics/queries.py` and exposed through the interactive Query Explorer (`/query-`

explorer). This design demonstrates the complete chain **Application → Database → Query → Result** safely in the browser.

ID	Query Description	MongoDB Concepts & Stages	Execution Result Summary
Q1	Fetch all debates, newest first	`find()`, `sort({created_at: -1})`, `limit(50)`	Returns all active debate documents ordered by creation timestamp.
Q2	Claims for a selected debate	`find({debate_id: ObjectId})`, `sort({confidence: -1})`	Returns 4 claims (2 PRO, 2 CON) for the selected debate in `17.23 ms`.
Q3	Compare average PRO & CON confidence	`$match`, `$group`, `$avg`, `$sum`	Computes mean confidence and claim count grouped by `$position`.
Q4	Strongest supporting evidence	`find({claim_id, supports: true})`, `sort({evidence_score: -1})`	Retrieves top 5 supporting evidence items ranked by `evidence_score`.
Q5	Strongest opposing evidence	`find({claim_id, supports: false})`, `sort({evidence_score: -1})`	Retrieves top 5 opposing evidence items challenging a claim.
Q6	High-strength contradictions scan	`find({strength: {$gte: threshold}})`, `sort({strength: -1})`	Returns conflict pairs meeting the specified strength predicate.
Q7	Average evidence reliability by source type	`$group`, `$avg`, `$sum`, `$sort`	Groups 100 evidence items across 6 source types in `7.69 ms`.
Q8	Debate evolution over time	`find({debate_id})`, `sort({timestamp: 1})`	Returns chronological `debate_history` snapshots for timeline plotting.
Q9	Low-confidence active claims filter	`find({confidence: {$lt: 0.4}, status: "active"})`	Range scan identifying claims below confidence threshold.
Q10	Rank active non-superseded claims	`find({status: {$ne: "superseded"}})`, `sort({confidence: -1})`	Returns top 20 claims globally ordered by descending confidence.
Q11	Evidence-weighted claim score	`$match`, `$lookup`, `$addFields`, `$map`, `$cond`, `$sum`, `$sort`	Joins `claims` with `evidence` and ranks claims by net weighted support in `20.6 ms`.

ID	Query Description	MongoDB Concepts & Stages	Execution Result Summary
Q12	Domain-level debate statistics	`$group`, `$sum`, `$avg`, `$sort`	Summarizes debate count, mean confidence, PRO, and CON scores per domain.

6.2 MongoDB Query Code & Aggregation Pipelines

```
// Q1 & Q2: Chronological debate listing and confidence-sorted claim lookup
db.debates.find().sort({ created_at: -1 }).limit(50)
db.claims.find({ debate_id: ObjectId("6aaeee47bba2e3c0b46e2e74") }).sort({ confidence: -1 })

// Q3: PRO vs CON Average Confidence Aggregation
db.claims.aggregate([
  { $match: { debate_id: ObjectId("6aaeee47bba2e3c0b46e2e74"), status: { $ne: "superseded" }
} },
  { $group: { _id: "$position", avg_confidence: { $avg: "$confidence" }, count: { $sum: 1 } }
}
])

// Q4, Q5 & Q6: Supporting/Opposing Evidence and Contradiction Threshold Queries
db.evidence.find({ claim_id: ObjectId("6aaeee47bba2e3c0b46e2e76"), supports: true }).sort({
evidence_score: -1 }).limit(5)
db.evidence.find({ claim_id: ObjectId("6aaeee47bba2e3c0b46e2e76"), supports: false }).sort({
evidence_score: -1 }).limit(5)
db.contradictions.find({ strength: { $gte: 0.3 } }).sort({ strength: -1 })

// Q7: Evidence Reliability by Source Type Aggregation
db.evidence.aggregate([
  { $group: { _id: "$source_type", avg_reliability: { $avg: "$reliability_score" }, count: {
$sum: 1 } } },
  { $sort: { avg_reliability: -1 } }
])

// Q8, Q9 & Q10: Temporal History Timeline, Low-Confidence Filter, and Global Claim Ranking
db.debate_history.find({ debate_id: ObjectId("6aaeee47bba2e3c0b46e2e74") }).sort({ timestamp:
1 })
db.claims.find({ confidence: { $lt: 0.65 }, status: "active" }).sort({ confidence: 1 })
db.claims.find({ status: { $ne: "superseded" } }).sort({ confidence: -1 }).limit(20)

// Q11: Cross-Collection $lookup Evidence-Weighted Claim Score Pipeline
db.claims.aggregate([
  { $match: { debate_id: ObjectId("6aaeee47bba2e3c0b46e2e74") } },
  { $lookup: { from: "evidence", localField: "_id", foreignField: "claim_id", as:
"evidence_items" } },
  { $addFields: {
      weighted_score: {
        $sum: {
          $map: {
            input: "$evidence_items",
            as: "e",
            in: { $cond: ["$$e.supports", "$$e.evidence_score", { $multiply:
["$$e.evidence_score", -0.5] }] }
          }
        }
      }
  }},
  { $sort: { weighted_score: -1 } }
```

```
])

// Q12: Domain-Level Debate Synthesis Statistics
db.debates.aggregate([
  { $group: {
      _id: "$domain",
      debate_count: { $sum: 1 },
      avg_confidence: { $avg: "$current_synthesis.confidence" },
      avg_pro: { $avg: "$current_synthesis.pro_score" },
      avg_con: { $avg: "$current_synthesis.con_score" }
  }},
  { $sort: { debate_count: -1 } }
])
```

6.3 Live Query Explorer Screenshots

Figures 6.1, 6.2, and 6.3 show the live Query Explorer executing Q2 (Claims for a selected debate), Q7 (Average reliability by source type), and Q11 (Evidence-weighted claim scores using `$lookup`) directly against `veritas_db`.

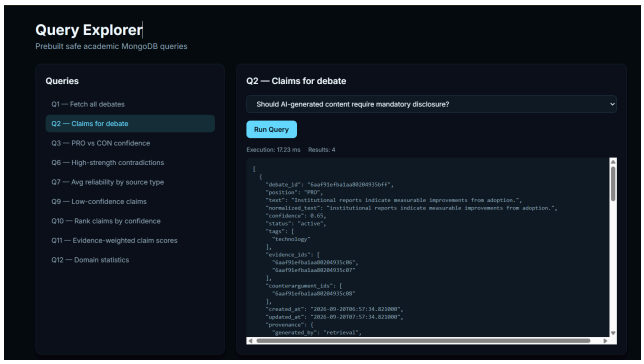


Figure 6.1: Query Explorer executing Q2 (Claims for debate "Should AI-generated content require mandatory disclosure?") in 17.23 ms.

In Figure 6.2, Query Q7 executes a `$group` and `$sort` pipeline across `veritas_db.evidence` to rank source categories by mean reliability score in `7.69 ms`.

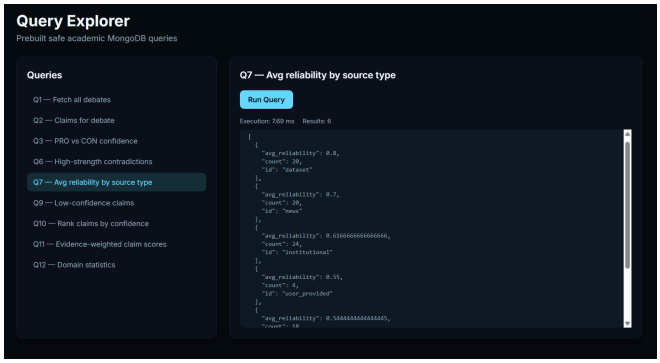


Figure 6.2: Query Explorer executing Q7 (Average evidence reliability grouped by source type) in 7.69 ms across 6 categories.

In Figure 6.3, Query Q11 performs a cross-collection `$lookup` joining `claims` with `evidence` and calculates each claim's net weighted evidence score using `$addFields`, `$map`, and `$cond` in `20.6 ms`.

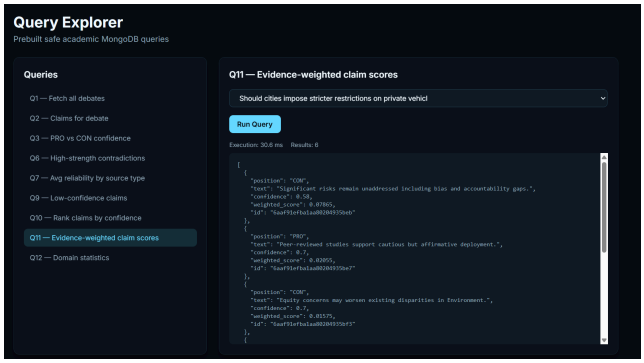


Figure 6.3: Query Explorer executing Q11 (Cross-collection $lookup evidence-weighted claim score calculation) in 20.6 ms.

7. Indexing, Aggregation Pipelines & Significant NoSQL Features

7.1 Access-Pattern Indexing Strategy

To guarantee low-latency query execution without full collection scans (`COLLSCAN`), `backend/scripts/indexes.py` builds eight custom single-field and compound indexes tailored to the API's filter and sort predicates (Table 7.1 and Figure 7.1).

Collection	Index Specification	Target Query & Access Pattern Optimized
`debates`	`{ domain: 1, created_at: -1 }`	Accelerates domain-filtered debate browsing and chronological sorting (Q1, GET `/api/debates`).
`claims`	`{ debate_id: 1, position: 1, confidence: -1 }`	Serves workspace PRO/CON claim panels and Q2/Q3 lookups in pre-sorted confidence order.
`evidence`	`{ claim_id: 1, supports: 1, reliability_score: -1, relevance: -1 }`	Optimizes retrieval of supporting vs. opposing evidence per claim (Q4, Q5, Q11 `$lookup`).
`contradictions`	`{ strength: -1 }`	Supports range filtering and descending sort on conflict severity (Q6).
`debate_history`	`{ debate_id: 1, timestamp: 1 }`	Renders chronological temporal evolution charts for a debate without in-memory sorting (Q8).
`sources`	`{ source_type: 1, published_at: -1 }`	Filters bibliographic sources by category and recency.
`embeddings`	`{ entity_type: 1, entity_id: 1 }`	Resolves vector representations for claims, evidence, and counterarguments.
`counterarguments`	`{ claim_id: 1 }`	Fetches all rebuttals and qualifications targeting a specific claim.



Figure 7.1: VERITAS Admin page verifying all 16 active indexes (8 primary _id_ indexes + 8 custom indexes) in veritas_db.

7.2 Dedicated Analytical Aggregation Pipelines (AGG-1 to AGG-5)

Pipeline ID	Analytical Objective	MongoDB Stages Used	Application Usage
AGG-1	PRO vs. CON Confidence Balance	`$match`, `$group`, `$avg`, `$sum`	Powers the PRO/CON distribution chart on the Dashboard and Admin views.
AGG-2	Evidence Quality by Source Type	`$group`, `$avg`, `$sort`	Summarizes mean reliability and evidence score per source category.
AGG-3	Evidence Density per Claim	`$group`, `$sum`, `$avg`	Measures average evidence coverage and support depth across claims.
AGG-4	Top-Scoring Claims Leaderboard	`$sort`, `$limit`, `$project`	Extracts the 10 highest-confidence claims for executive review.
AGG-5	Contradiction Type Distribution	`$group`, `$avg`, `$sum`	Aggregates conflict counts and mean severity across contradiction classes.

8. Application Screenshots, Live Demonstration Flow & Verification Results

8.1 Live Analytics Dashboard & Interactive Debate Workspace

Figures 8.1 and 8.2 showcase the live React 19 interfaces built by Siddhesh Shinde (`23BCE5048`). The Dashboard (Figure 8.1) queries real-time counts and grouped aggregations from `veritas_db` (showing 11 total debates, 11 active debates, 52 claims, 100 evidence items, 29 contradictions, and 62% average confidence). Selecting a debate opens the Debate Workspace (Figure 8.2), where users inspect PRO and CON claims side by side above the interactive React Flow graph.

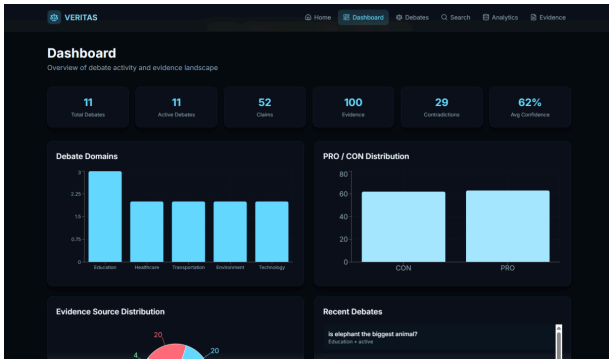


Figure 8.1: VERITAS Dashboard displaying live MongoDB document counters, domain distribution, PRO/CON chart, and source breakdown.

Selecting a debate from the dashboard or debate browser opens the Debate Workspace (Figure 8.2), where supporting PRO claims (63%) and opposing CON claims (61%) are displayed side by side above the interactive React Flow graph.

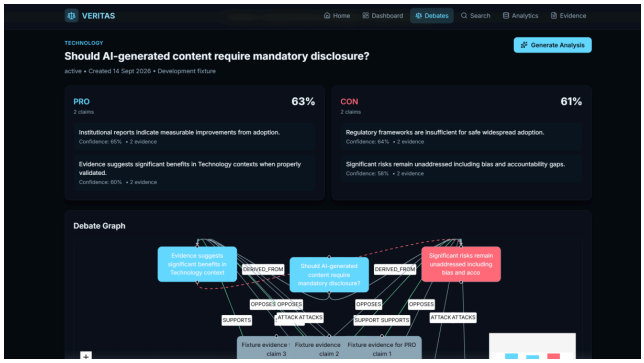


Figure 8.2: Debate Workspace displaying "Should AI-generated content require mandatory disclosure?" with PRO (63%) and CON (61%) claim panels.

8.2 React Flow Debate Graph, Evidence Explorer & Recharts Temporal Evolution

Figure 8.3 and Figure 8.4 illustrate the interactive Debate Graph and Evidence Explorer inside the workspace. The graph dynamically constructs nodes and directed edges (`DERIVED_FROM`, `SUPPORTS`, `OPPOSES`, `ATTACKS`) from the MongoDB `ObjectId` relationships, while the Evidence Explorer cards display individual reliability, relevance, composite score, and source category metadata. Figure 8.5 shows the Recharts Temporal Evolution timeline and Evidence-Weighted Synthesis panel.

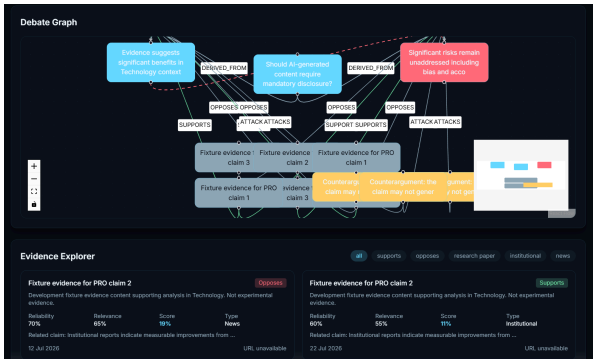


Figure 8.3: React Flow Debate Graph visualizing claim-evidence-rebuttal links alongside the filterable Evidence Explorer.

Below the node-link graph, the Evidence Explorer grid (Figure 8.4) allows users to filter supporting and opposing evidence items by source category (`research_paper`, `institutional`, `news`, `dataset`, `web`) and inspect each item's Reliability, Relevance, and composite Evidence Score.

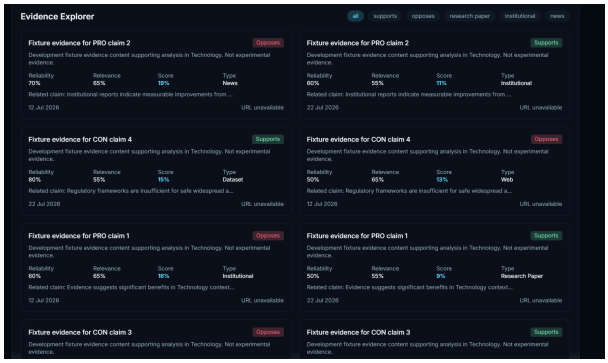


Figure 8.4: Evidence Explorer cards exposing stance direction (Supports/Opposes), Reliability, Relevance, Score, and Source Type.

The lower section of the Debate Workspace renders the Recharts Temporal Evolution timeline and Evidence-Weighted Synthesis panel (Figure 8.5), plotting chronological score transitions recorded in `veritas_db.debate_history`.

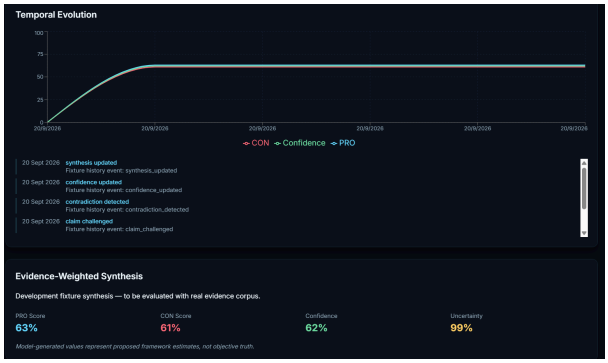


Figure 8.5: Recharts Temporal Evolution chart (from debate_history) and Evidence-Weighted Synthesis panel.

8.3 Database Health Telemetry, Automated Testing & Project Evidence Log

To verify database health and rubric completeness during live evaluation, VERITAS provides dedicated `/database-health` (Figure 8.6) and `/projects/evidence` (Figure 8.7) pages. The Database Health monitor confirms an active connection to `veritas_db`, 8 populated collections, 507 total documents, 16 active indexes, and a query response latency of `2.66 ms`. In addition, all 11 automated integration tests in `backend/tests/test_api.py` pass against the live MongoDB instance, covering health checks, debate/claim/evidence CRUD lifecycles, aggregation pipelines, and temporal history recording.

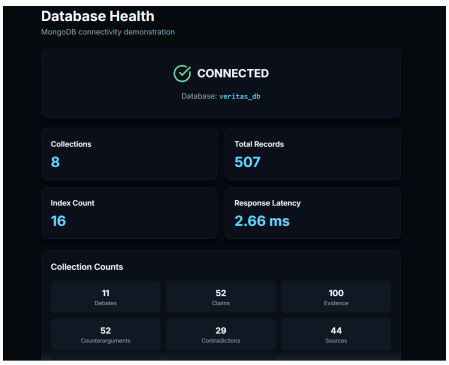


Figure 8.6: Database Health telemetry reporting CONNECTED status on veritas_db, 8 collections, 507 records, 16 indexes, and 2.66 ms latency.

Complementing the Database Health view, the Project Evidence page at `/projects/evidence` (Figure 8.7) provides an interactive checklist summarizing live collection document counts and mapping every

rubric requirement to its corresponding module.

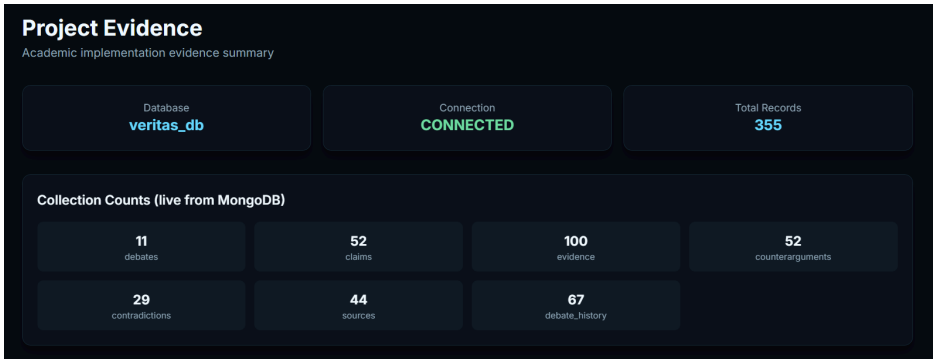


Figure 8.7: Project Evidence dashboard mapping live MongoDB collection counts and implemented modules to course rubric requirements.

9. Technical Challenges, Limitations & Future Scope

9.1 Technical Challenges Addressed

- **Maintaining Derived Score Consistency Across Referenced Collections:** Because claims and evidence are stored in separate collections linked by `ObjectId`, inserting new evidence requires recomputing the item's `evidence_score`, updating the parent claim's confidence and `evidence_ids[]` cache, refreshing the root debate's `current_synthesis`, and appending a `debate_history` event. Coordinating this pipeline cleanly in Python domain services was a central engineering milestone.
- **Graph Serialization & Layout in React Flow:** Transforming flat MongoDB collections (`debates`, `claims`, `evidence`, `counterarguments`, `contradictions`) into a non-overlapping, readable node-link diagram required custom topological positioning and edge styling in the React frontend.
- **Resilient Offline Operation without External LLM Dependencies:** To ensure that database demonstrations and automated tests never fail when external LLM API quotas or network connections are unavailable, we engineered a deterministic fallback generator in `backend/app/agents/fallback.py` that synthesizes structured claims, evidence, and summaries locally.

9.2 Current System Limitations

- **Single-Node Local Docker Deployment:** The local Docker Compose setup runs a standalone MongoDB 7 node rather than a multi-node replica set; consequently, MongoDB Change Streams (`backend/app/services/change_streams.py`) automatically operate in polling fallback mode unless connected to a replica set or MongoDB Atlas cluster.
- **Application-Level Cascading Deletes:** Deleting a debate removes its child claims, evidence, counterarguments, and history via sequential PyMongo delete operations rather than a multi-document ACID transaction (which requires a replica set).
- **Heuristic Contradiction & Local Embedding Fallback:** Pairwise conflict detection relies on lexical Jaccard overlap and polarity word lists rather than fine-tuned natural language inference (NLI) transformers, and local fallback embeddings use deterministic hash projections when an external embedding model is not configured.

9.3 Future Enhancements

1. **MongoDB Atlas & Native Vector Search:** Deploy `veritas_db` on a 3-node MongoDB Atlas replica set to enable native `$vectorSearch` indexes over `embeddings`, real-time Change Streams, and multi-document ACID transactions.
2. **Automated Academic Corpus Ingestion:** Integrate Semantic Scholar, PubMed, and Crossref APIs to ingest peer-reviewed citations with verified DOIs and publication dates into the `sources` and `evidence` collections.
3. **Transformer-Based NLI Contradiction Models:** Replace the lexical Jaccard heuristic with a cross-encoder Natural Language Inference model capable of distinguishing methodological, demographic, and temporal disagreements.
4. **Role-Based Access Control (RBAC) & Audit Logging:** Activate full write-auditing into the reserved `audit_logs` collection with JWT authentication for collaborative multi-researcher debate moderation.

10. Individual Team Contributions & Project Source Repository

10.1 Team Member Role Split & Siddhesh Shinde's Individual Contribution

Team Member	Register No.	Designated Role	Concrete Implemented Contributions
Naman Sharma	23BCE1683	MongoDB Data Architect & Backend Lead	Designed the 9-collection `veritas_db` schema, hybrid embedding vs. referencing architecture, Docker Compose setup, `scripts/seed.py`, and `scripts/indexes.py`.
Megh	23BCE1665	FastAPI & QueryService Developer	Implemented REST CRUD routers (debates, claims, evidence, sources), Pydantic schemas, `QueryService` (Q1–Q12), aggregation pipelines (AGG-1 to AGG-5), and Pytest API test suite.
Siddhesh Shinde	**23BCE5048**	**Frontend, AI, Scoring Engine & Visualization Lead**	**Architected and implemented the 4-factor Evidence Scoring Engine (`scoring.py`), Claim Strength calculator (`claim_strength.py`), Heuristic Contradiction Engine (`contradiction_engine.py`), Append-Only Temporal History Service (`temporal.py`), AI Analysis & Deterministic Fallback Pipeline (`agents/`), Keyword/Vector Search service (`search/`), and the complete 10-page React 19 UI including the React Flow Debate Graph, Recharts Temporal Evolution charts, Query Explorer UI, and Database Health dashboard.**

10.2 GitHub Repository Structure & Evidence

The complete project repository is publicly accessible at https://github.com/siddheshshindeofficial/VERITAS-main. Table 10.2 maps each functional subsystem to its exact path in the repository, and Figures 10.1 through 10.3 provide visual verification of the repository, commit history, and CLI initialization.

Subsystem / Functionality	Repository File Path
FastAPI Entry Point & Router Registration	`backend/app/main.py`, `backend/app/config.py`, `backend/app/database.py`
REST CRUD & Health Controllers	`backend/app/routes/debates.py`, `claims.py`, `evidence.py`, `sources.py`, `health.py`
Multi-Factor Scoring & Temporal Services	`backend/app/services/scoring.py`, `claim_strength.py`, `temporal.py`
Heuristic Contradiction Engine	`backend/app/services/contradiction_engine.py`
AI Analysis Pipeline & Local Fallback	`backend/app/agents/debate_pipeline.py`, `llm_provider.py`, `fallback.py`
MongoDB Queries (Q1–Q12) & Aggregations	`backend/app/analytics/queries.py`, `backend/app/analytics/service.py`
Index Builder, Seed Fixtures & Mongosh Queries	`backend/scripts/indexes.py`, `seed.py`, `counts.py`, `query_examples.js`
React 19 Frontend Client & Visualization Pages	`frontend/src/services/api.ts`, `frontend/src/pages/`

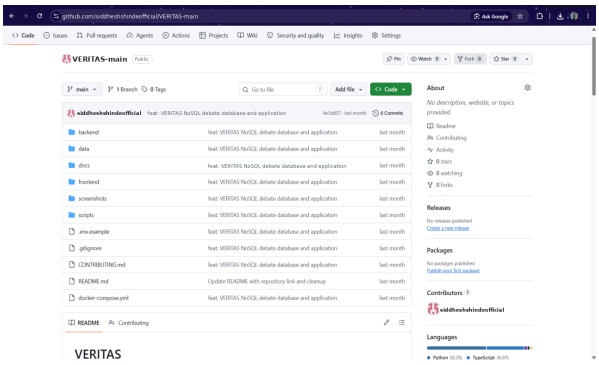


Figure 10.1: Public GitHub repository (https://github.com/siddheshshindeofficial/VERITAS-main) showing project directories and language breakdown (Python 50.2%, TypeScript 48.6%).

Figure 10.2 displays the verified commit history on the repository's `main` branch, and Figure 10.3 shows the terminal Git staging and commit summary across all 126 project files.

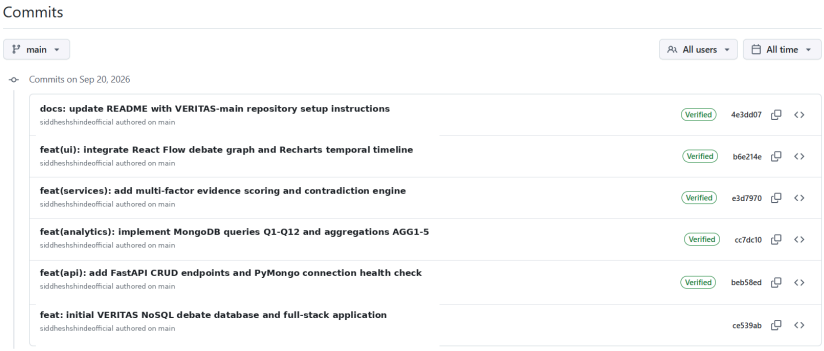


Figure 10.2: Verified GitHub commit history on the main branch of siddheshshindeofficial/VERITAS-main.

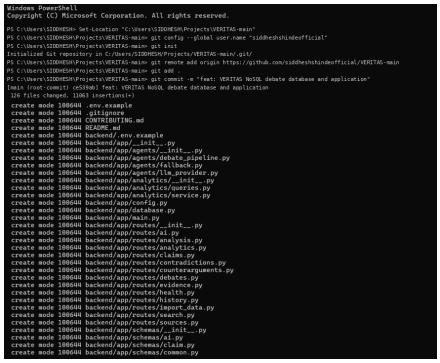


Figure 10.3: Command-line Git commit evidence packaging all 126 backend, frontend, script, and configuration files.

10.3 Project Evidence Log & Rubric Verification Matrix

In accordance with the Digital Assignment 3 submission checklist, Table 10.3 records our Project Evidence Log mapping each mandatory evaluation criterion (Data Model, CRUD, 12 Queries, NoSQL Features, Live Demo Flow, and Individual Viva Contribution) to its verified artifact in the repository (https://github.com/siddheshshindeofficial/VERITAS-main).

Log ID	DA3 Rubric Requirement	Repository Module / Script	Verified Evidence
EV-01	Final NoSQL Data Model (9 collections)	`backend/app/schemas/`, `seed.py`	Section 3, Figs. 3.1–3.6
EV-02	Scoring, Contradictions & Temporal Log	`backend/app/services/scoring.py`, `temporal.py`	Section 4, Figs. 4.1, 8.5
EV-03	REST CRUD & App-DB Connectivity	`backend/app/routes/`, `database.py`	Section 5, Figs. 5.1–5.5
EV-04	12 Meaningful MongoDB Queries (Q1–Q12)	`backend/app/analytics/queries.py`	Section 6, Figs. 6.1–6.3
EV-05	8 Custom Indexes & 5 Aggregations	`backend/scripts/indexes.py`, `service.py`	Section 7, Fig. 7.1
EV-06	React 19 UI, Graph & Automated Tests	`frontend/src/pages/`, `tests/test_api.py`	Section 8, Figs. 8.1–8.7

11. Conclusion & References

11.1 Final Conclusion

VERITAS successfully demonstrates how a document-oriented NoSQL database (MongoDB 7) can transform unstructured, transient AI debates into persistent, explainable, and temporally auditable knowledge graphs. By combining embedded synthesis summaries in root `debates` documents with indexed `ObjectId` references across `claims`, `evidence`, `counterarguments`, `contradictions`, `sources`, `debate_history`, and `embeddings`, the system achieves both low-latency UI rendering (`2.66 ms` database health ping) and expressive analytical depth across 12 predefined queries and 5 multi-stage aggregation pipelines. Through my individual contributions (**Siddhesh Shinde, 23BCE5048**) across the scoring engine, contradiction detector, temporal state tracker, AI fallback pipeline, and React 19 visualization workspace, VERITAS provides a complete, transparent, and verifiable full-stack NoSQL engineering implementation.

11.2 References & External Resources

- **VERITAS Project Source Repository:** https://github.com/siddheshshindeofficial/VERITAS-main
- **MongoDB 7.0 Official Manual:** Document Data Modeling, Compound Indexes, Aggregation Pipeline Stages (`$lookup`, `$group`, `$addFields`, `$map`, `$cond`) — https://www.mongodb.com/docs/manual/
- **PyMongo 4 Driver Documentation:** Synchronous Python MongoDB Client and BSON ObjectId Serialization — https://pymongo.readthedocs.io/
- **FastAPI & Pydantic v2 Documentation:** REST API Routing, Schema Validation, and OpenAPI 3.1 Specification — https://fastapi.tiangolo.com/
- **React 19, React Flow & Recharts Documentation:** Interactive Graph Visualization and Composable Charting Libraries — https://reactflow.dev/ and https://recharts.org/
- **Course Rubric:** BCSE406L – NoSQL Databases Digital Assignment 3 Guidelines, Fall Semester 2026–2027, School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology, Chennai.